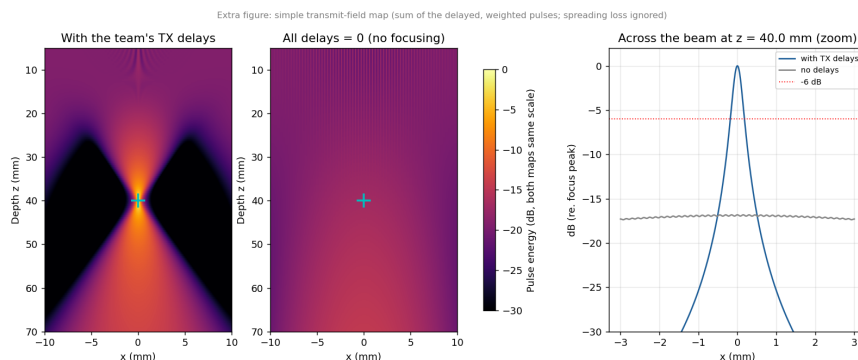


Team 2: Transmit (TX) Beamforming

When should each of the 128 elements fire, and how strongly, so that the sound meets at the focus?

NITK-UsoundSim · 2-D ultrasound simulator · explanation, input, output and run guide

- Input: probe file (128 elements, 0.30 mm pitch, 4 to 12 MHz, 8 MHz centre, PZT-5H) and beam file (focus $x = 0$ mm, $z = 40$ mm, $c = 1540$ m/s, Hann weights).
- Output: one delay and one weight per element. Delays go from 0 us (edge elements) to 2.7950 us (centre elements).
- Hand-off: tx_beamforming_package.npz is loaded by the Acoustic Propagation team.
- The team's 2 tests pass (RESULT 2/2 tests passed).
- Run it: `python3 run_team2.py` (Windows: `python run_team2.py`). About 20 seconds.



Where the 128 delayed pulses add up (extra figure made by the runner). Left: with the team's delays the energy gathers at the focus (cyan +). Middle: with all delays zero there is no focus.

Contents

1 Overview	3
Files in this folder	3
2 Theory	4
2.1 Element positions of a linear array	4
2.2 Why transmit focusing works	4
2.3 Steering delays	4
2.4 Apodization: main lobe vs side lobes	5
2.5 The Gaussian pulse	5
2.6 Grating lobes (why pitch matters)	5
2.7 What the TX package contains	5
3 Code explanation	6
3.1 config/transducer_interface.json and config/beamforming.json	6
3.2 src/transducer_interface.py	6
3.3 src/beamformer.py	6
3.4 src/acoustic_handoff.py	7
3.5 src/__init__.py	7
3.6 run_transmit_beamforming.py (step 1)	7
3.7 visualize_transmit_beamforming.py (step 2)	7
3.8 tests/test_beamformer.py	7
3.9 code/docs/	7
3.10 How run_team2.py uses the team's own code	8
3.11 run_team2.py step by step (written for this demo, not team code)	9
3.12 Full listings	9
4 Input	20
input/transducer_interface.json (probe)	20
input/beamforming.json (beam settings)	20
5 Output	21
5.1 Data files (made by the team's code)	21
5.2 Figures made by the team's script	21
5.3 Extra figures made by run_team2.py (using the team's functions and outputs)	22
5.4 Console output (real, from output/console_log.txt)	23
5.5 What the results tell us	26
6 How to run	27
Troubleshooting	27
7 Check yourself	28

1 Overview

What Team 2 does and what is in the folder

An ultrasound probe is a row of many small crystals (**elements**). Each element can send a short sound pulse. If all elements fire at the same time, the sound goes straight down as a wide beam. If we fire them at slightly **different times**, we can make all the waves meet at one chosen point, the **focus**. There the sound is strong and the beam is narrow, which gives a sharper image.

Team 2's code calculates these firing times (**transmit delays**) and the strength of each element (**apodization weights**) for a 128-element linear probe. It also builds the delayed pulse of every element, and saves everything in a **TX package** file for the next team.

Place in the pipeline

Transducer (Team 1: probe properties) → **TX Beamforming (Team 2: delays + weights)** → Acoustic Propagation (Team 3: the waves travel into the body).

Note about code/docs/*.py

The seven .py files in code/docs/ are byte-identical copies of Team 1's (Transducer) piezo-material code. They only import each other. Nothing in Team 2's src/, scripts or tests imports them, so they are reference copies and are not run by the demo.

Files in this folder

File / folder	What it is
run_team2.py	The one command to run: team scripts + tests, summary, extra figures, all outputs.
README.md, requirements.txt	Short run instructions. Packages needed: numpy, matplotlib. Also pytest, only if you want to run code/tests yourself.
.vscode/settings.json	VS Code settings so the team's files run with the Run button.
code/	The team's ORIGINAL code, copied unchanged (same layout as the team's Usound/ folder).
code/src/beamformer.py	The main maths: element positions, delays, apodization, Gaussian pulse, TX package, saving.
code/src/transducer_interface.py	Reads the probe file and checks the values (128 elements, frequency range).
code/src/acoustic_handoff.py	Loader that the next team uses to read the TX package safely.
code/run_transmit_beamforming.py	Team's main script: builds and saves the TX package and pulses.
code/visualize_transmit_beamforming.py	Team's plot script: two delay plots and a printed summary.
code/tests/test_beamformer.py	Team's two tests (shapes, delays).
code/config/*.json	Team's original settings (same as input/).
code/docs/	RUN_ORDER.txt, ACOUSTIC_HANDOFF.md, TRANSDUCER_INPUTS.md, plus 7 .py files (see note below).
code/outputs/	Outputs the team saved earlier (kept for reference).
input/, output/	The 2 input files used by the run; everything produced by run_team2.py.

2 Theory

The ideas behind the code, in easy words

2.1 Element positions of a linear array

The 128 elements sit in a straight line along x, equally spaced by the **pitch** $p = 0.30$ mm. The code puts the centre of the array at $x = 0$, so element n (counting from 0) is at

$$x_n = \left(n - \frac{N-1}{2}\right)p, \quad n = 0, 1, \dots, N-1, \quad N = 128$$

So the elements go from -19.05 mm ... 19.05 mm. The distance from the first to the last element centre (the **aperture**) is 38.10 mm. Depth into the body is called z .

2.2 Why transmit focusing works

Think of runners who start at different places but must all reach the finish line (the focus) at the same moment. Runners who are farther away must start earlier. For the probe: the **edge elements** are farther from a focus in front of the centre, so they **fire first**; the **centre elements** are closest, so they **fire last**. All the wavefronts then arrive at the focus together and add up (constructive interference). Everywhere else they arrive at different times and partly cancel.

The distance from element n to the focus (x_f, z_f) is

$$d_n = \sqrt{(x_n - x_f)^2 + z_f^2}$$

The farthest element fires at time 0. Every other element waits for the extra travel time it saves:

$$\tau_n = \frac{\max_k(d_k) - d_n}{c}$$

c = speed of sound (1540 m/s in soft tissue). All delays are 0 or positive.

Worked example (numbers from this run)

Edge element: $x = 19.05$ mm, focus depth 40 mm. $d = \sqrt{(19.05^2 + 40^2)} = 44.305$ mm.

Centre elements: $d \approx 40$ mm. Extra path = 4.305 mm.

Delay of the centre = 4.305 mm / 1540 m/s ≈ 2.795 us. The run gives 2.7950 us, the same.

2.3 Steering delays

To tilt the beam by an angle θ without focusing, each element is delayed in proportion to its position (a straight ramp). The code then shifts the ramp so the smallest delay is 0:

$$\tau_n = \frac{x_n \sin \theta}{c} - \min_k \left(\frac{x_k \sin \theta}{c} \right)$$

In the team's code, when the steering angle is not 0 the delays are **only** this ramp (a tilted plane wave); the focus point is not used in that case. With the default setting (0 degrees) the focusing formula of 2.2 is used.

2.4 Apodization: main lobe vs side lobes

Apodization means giving each element a different strength (weight). With equal weights (**rect**) the beam has the narrowest main lobe but strong **side lobes** (unwanted weaker beams to the side that create clutter). With a smooth window that is strong in the middle and weak at the edges (**Hann, Hamming**) the side lobes drop a lot, but the main lobe gets a bit wider. This is the main-lobe / side-lobe trade-off.

$$w_n^{\text{Hann}} = 0.5 - 0.5 \cos \frac{2\pi n}{N-1}, \quad w_n^{\text{Hamming}} = 0.54 - 0.46 \cos \frac{2\pi n}{N-1}$$

The code uses numpy's hanning/hamming and divides by the maximum, so the biggest weight is 1.

Window	-6 dB main-lobe width	Highest side lobe
rect	0.340 deg	-13.3 dB
hann	0.570 deg	-31.5 dB
hamming	0.520 deg	-42.6 dB

Values measured by `run_team2.py` from `make_apodization` (continuous-wave beam pattern at 8 MHz).

2.5 The Gaussian pulse

Each element sends a short burst: a sine wave at the centre frequency $f_0 = 8$ MHz, multiplied by a bell-shaped (Gaussian) envelope so it starts and ends smoothly:

$$p(t) = e^{-t^2/(2\sigma^2)} \sin(2\pi f_0 t), \quad \sigma = \frac{\sigma_{\text{cycles}}}{2\pi f_0}$$

With `sigma_cycles = 1` (from the beam file), $\sigma = 1/(2\pi \cdot 8 \text{ MHz}) \approx 19.9 \text{ ns}$. This is a very short pulse (about one oscillation), so its spectrum is wide: the run measured peak at 9.57 MHz, -6 dB band 3.11 ... 18.05 MHz. A short pulse gives good depth (axial) resolution. Element n sends $w_n \cdot p(t - \tau_n)$.

2.6 Grating lobes (why pitch matters)

The wavelength is $\lambda = c/f_0 = 1540/8 \cdot 10^6 = 0.1925 \text{ mm}$. The pitch (0.30 mm) is 1.56λ , more than $\lambda/2$. Then extra copies of the main beam, called **grating lobes**, can appear at

$$\sin \theta_g = \frac{\lambda}{p} \Rightarrow \theta_g \approx 39.9^\circ$$

The team marks the 0.30 mm pitch as an editable assumption because Philips does not publish it.

2.7 What the TX package contains

Field	Shape	Meaning
<code>element_x_m</code>	(128,)	x position of each element centre, metres
<code>tx_delays_s</code>	(128,)	firing delay of each element, seconds (the key output)
<code>tx_weights</code>	(128,)	apodization weight of each element (0 to 1)
<code>tx_frequency_hz</code>	scalar	centre frequency (8e6)
<code>sound_speed_m_s</code>	scalar	speed of sound used (1540)
<code>focus_x_m, focus_z_m</code>	scalar	focus point (0, 0.04)
<code>steering_angle_deg</code>	scalar	steering angle (0)

The next team must use the same element positions, ordering, sound speed and coordinates (x across, z into the body), as the team's README asks.

3 Code explanation

Every file of the team, section by section

Run order designed by the team (docs/RUN_ORDER.txt): (1) edit config/transducer_interface.json if Team 1 gives real values, (2) edit config/beamforming.json for focus / steering / sound speed, (3) run run_transmit_beamforming.py, (4) give outputs/tx_beamforming_package.npz to Acoustic Propagation.

3.1 config/transducer_interface.json and config/beamforming.json

- **transducer_interface.json**: probe name, $n_elements = 128$, $pitch_m = 0.0003$, $frequency_min_hz = 4e6$, $frequency_max_hz = 12e6$, $center_frequency_hz = 8e6$, $element\ width/height = null$ (unknown), $material = PZT-5H$, and a note.
- **beamforming.json**: $focus_x_m = 0$, $focus_z_m = 0.040$, $steering_angle_deg = 0$, $sound_speed_m_s = 1540$, $apodization = hann$, $pulse_cycles = 2$, $pulse_duration_sigma_cycles = 1$.
- Honest note: **pulse_cycles** is not read by any of the team's code. Only $pulse_duration_sigma_cycles$ changes the pulse. $element_width_m$, $element_height_m$ and $material$ are stored but not used in the maths.

3.2 src/transducer_interface.py

- **@dataclass TransducerConfig**: a simple container with one field per probe value. The two size fields may be None (the `float | None` style needs Python 3.10+).
- **from_json(path)**: reads the JSON file, checks that the six required fields exist (else `ValueError 'Missing transducer fields'`), keeps only known fields, builds the object and calls `validate()`.
- **validate()**: stops with an error if $n_elements$ is not exactly 128, if $pitch$ is not positive, if the frequency range is wrong, or if the centre frequency is outside the 4 to 12 MHz range.

3.3 src/beamformer.py

- **create_linear_array(n, pitch)**: returns $x_n = (n - (N-1)/2) \cdot pitch$, a centred row of positions (formula 2.1).
- **make_apodization(n, kind)**: 'none'/'uniform'/'rect' gives all ones; 'hann'/'hanning' uses `np.hanning`; 'hamming' uses `np.hamming`; anything else raises `ValueError`. Result is divided by its maximum. Note: `np.hanning` is exactly 0 at both ends, so elements 1 and 128 get weight 0 (they 'fire' with zero strength).
- **calculate_tx_delays(x, xf, zf, c, angle)**: first checks $c > 0$ and $zf > 0$. If the angle is (almost) 0 it uses the focusing formula of 2.2; otherwise the steering ramp of 2.3, shifted so the minimum is 0.
- **gaussian_pulse(t, f0, sigma_cycles)**: the pulse of 2.5.
- **generate_element_pulses(delays, f0, weights, fs, sigma_cycles)**: sampling rate $fs = \max(20 \cdot f0, 100 \text{ MHz}) = 160 \text{ MHz}$ here; time axis from $-dur$ to $max_delay + dur$ with $dur = \max(6 \cdot sigma_cycles / f0, 5 / f0) = 0.75 \text{ us}$; row n is $w_n \cdot p(t - \tau_n)$. Returns the time axis and a 128-row array.
- **build_tx_package(transducer, beam_cfg)**: joins everything: positions, delays, weights, and the scalars, into one dictionary (the TX package of 2.7).
- **save_package**: `np.savez` to a .npz file. **save_summary**: writes a small JSON (probe, pitch, $f0$, focus, steering, c , min/max delay in s and max delay in us).

3.4 src/acoustic_handoff.py

- REQUIRED lists the five fields the next team must get.
- **load_tx_package(path)**: opens the .npz, raises an error if any required field is missing, copies all fields into a dictionary, and checks that element_x_m, tx_delays_s and tx_weights each have shape (128,).

3.5 src/__init__.py

Only a one-line description string. It makes src a package so that `from src.beamformer import ...` works.

3.6 run_transmit_beamforming.py (step 1)

- R = folder of the script, O = R/outputs (created if missing).
- Reads the probe file with `TransducerConfig.from_json` (with checks) and the beam file with `json`.
- `build_tx_package` → `save_package` (tx_beamforming_package.npz) → `save_summary` (tx_beamforming_summary.json).
- `generate_element_pulses` with the package's delays, frequency and weights → `tx_element_pulses.npz` (time_s, pulses).
- Prints: completed, number of elements, frequency, delay range in us, and the path of the hand-off file.

3.7 visualize_transmit_beamforming.py (step 2)

- Loads `outputs/tx_beamforming_package.npz` using a relative path, so it must be run from the team folder.
- Converts to easy units: element number 1 to 128, x in mm, delay in us.
- Figure 1: delay vs element number. Figure 2: delay vs element position (mm). Each ends with `plt.show()`.
- Prints a summary: elements, frequency, focus, sound speed, steering, min and max delay.
- In `run_team2.py` each `plt.show()` is replaced by 'save as PNG' (the team file itself is not changed).

3.8 tests/test_beamformer.py

- **test_shapes**: the array has 128 positions centred on 0; the package's delays and weights have shape (128,).
- **test_delay**: for a focus at 40 mm with $c = 1540$ m/s, all delays are ≥ 0 and the smallest is 0.
- The team wrote these for `pytest`. `run_team2.py` does not need `pytest`: it imports the module and calls each `test_*` function directly, printing PASS/FAIL. To use `pytest` yourself: `cd code && python3 -m pytest tests` (`pytest` is listed in `requirements.txt`).

3.9 code/docs/

- `RUN_ORDER.txt`, `ACOUSTIC_HANDOFF.md`, `TRANSDUCER_INPUTS.md`: short notes on run order, how the next team should use the package, and what the Transducer team must supply.
- `piezo.py`, `registry.py`, `exceptions.py`, `process.py`, `main.py`, `add_attributes.py`, `new_piezo.py`: exact copies of Team 1's piezo-material registry code (a `Piezo` class with a validated frequency range, a registry of materials, custom errors, and a demo `main.py`). Not imported by Team 2's code and not run here.

3.10 How run_team2.py uses the team's own code

All the transmit-beamforming logic comes from the team's existing code in code/. It is used **unchanged**: nothing was rewritten. run_team2.py is only a **demo wrapper** written for this demo. It copies code/ to a temporary folder, runs the team's scripts, functions and tests there, collects the results and adds some explanation figures.

Where run_team2.py uses your code

run_team2.py line(s)	Team file / function / script used	What it does there
316	the whole code/ folder	Copies the team's code unchanged into a temporary working folder (old code/outputs removed so all outputs are fresh).
318-319	config/transducer_interface.json, config/beamforming.json	Puts the two input/ files in place of the team's config files in the working copy.
328	code/run_transmit_beamforming.py	Runs the team's main script as it is: builds and saves the TX package, the summary and the element pulses.
94, 334	code/visualize_transmit_beamforming.py	Runs the team's plot script as it is (only plt.show() is swapped for 'save PNG').
100-105, 339	code/tests/test_beamformer.py (test_delay, test_shapes)	Imports the team's test file and calls each test_* function; prints PASS/FAIL.
345-346	files written by the team's code	Copies tx_beamforming_package.npz, tx_beamforming_summary.json and tx_element_pulses.npz to output/.
350-352	the same three output files	Reads the team's results to print the STEP 4 summary.
385-386	src/acoustic_handoff.py: load_tx_package()	Checks the saved package with the team's own loader.
164	src/beamformer.py	Imports three team functions for the extra figures.
186	calculate_tx_delays()	Example steering delays (-15 and +15 degrees) in extra_1.
199, 211	make_apodization()	rect / Hann / Hamming weights for extra_2.
230	tx_element_pulses.npz (team output)	Element signals plotted in extra_3.
234	gaussian_pulse()	The team's pulse, plotted with its spectrum in extra_3.
166	tx_beamforming_package.npz (team output)	Element positions, delays and weights used for all extra figures and the field map.

What run_team2.py adds on top (written for this demo, not the team's code)

run_team2.py line(s)	Demo addition	Purpose
49-57	class Tee	Prints to the screen and keeps a copy of all text for output/console_log.txt.
60-61	banner()	Prints the STEP headings.
64-75	run_script()	Runs a team script in the working folder, shows its output, stops on an error.
80-95	VIS_WRAPPER	Replaces plt.show() with 'save figure as PNG' (team_fig1..., team_fig2...) instead of opening windows.
98-110	TEST_WRAPPER	Runs the team's tests without pytest and prints a RESULT line.
113-131	console_to_png()	Draws the console text as output/console_screenshot.png.

run_team2.py line(s)	Demo addition	Purpose
137-140	pulse_pair_energy()	Demo formula: exact overlap integral of two of the team's Gaussian pulses shifted by dt.
143-159	field_energy_map()	Demo calculation: energy of the summed delayed, weighted pulses at every (x, z) point (spreading loss and directivity ignored).
162-297	make_extra_figures()	Makes extra_1 to extra_4 and the extra numbers (it calls the team functions listed above).
195-209	beam pattern (array factor)	Demo metric: -6 dB main-lobe width and highest side lobe for each window.
214, 245-246	grating-lobe angle	Demo formula $\text{asin}(\text{wavelength} / \text{pitch})$.
240-244	pulse spectrum (FFT)	Demo metric: spectrum peak and -6 dB band of the pulse.
260-296	field map and its numbers	Map with the team's delays vs. all delays 0; peak depth, -6 dB beam width, gain at the focus.
355-380	STEP 4 summary table	Extra derived values: aperture, wavelength, F-number, which elements fire first/last.
314-324	STEP 0	Temporary working copy; reads input/ and prints the input values.
416-423	console log + screenshot	Writes output/console_log.txt and output/console_screenshot.png at the end.

Honest note

The extra_figures and the numbers they print (beam-pattern widths, side lobes, spectrum, grating-lobe angle, field map, -6 dB width, gain) are **demo additions for illustration**. The formulas in pulse_pair_energy, field_energy_map and make_extra_figures are not the team's work; they only use the team's functions and outputs as input.

To see it yourself: open run_team2.py at the lines above; Cmd/Ctrl+Click a team function in VS Code opens the original file in code/.

3.11 run_team2.py step by step (written for this demo, not team code)

- Step 0: copies code/ to a temporary folder and puts input/*.json into its config/.
- Steps 1 to 3: runs the two team scripts and the tests in that folder, in the team's order.
- Step 4: copies the team's outputs to output/, prints a summary read from them, and checks the file with the team's own load_tx_package().
- Step 5: extra figures with the team's functions (make_apodization, gaussian_pulse, calculate_tx_delays). The field map adds up every element's delayed, weighted pulse at each grid point and shows the total pulse energy (it ignores spreading loss and element directivity, so it only shows where the pulses line up).
- Saves console_log.txt and console_screenshot.png.

3.12 Full listings

code/config/transducer_interface.json

```
1 {"probe_name": "Philips L12-4 reference model", "n_elements": 128, "pitch_m": 0.00030, "frequency_min_hz": 4000000.0, "frequency_max_hz": 12000000.0, "center_frequency_hz": 8000000.0, "element_width_m": null, "element_height_m": null, "material": "PZT-5H", "notes": "Pitch and centre frequency are editable simulation assumptions unless supplied by the project/transducer team."}
```

code/config/beamforming.json

```
1 {"focus_x_m":0.0,"focus_z_m":0.040,"steering_angle_deg":0.0,"sound_speed_m_s":1540.0,"apodization"
  ↳ : "hann","pulse_cycles":2.0,"pulse_duration_sigma_cycles":1.0}
```

code/src/__init__.py

```
1 """NITK 128-element transmit beamforming module."""
```

code/src/transducer_interface.py

```
1 from dataclasses import dataclass
2 from pathlib import Path
3 import json
4 @dataclass
5 class TransducerConfig:
6     probe_name:str; n_elements:int; pitch_m:float; frequency_min_hz:float; frequency_max_hz:float;
7     ↳ center_frequency_hz:float; element_width_m:float|None=None; element_height_m:float|None=None; mat
8     ↳ erial:str=""
9     @classmethod
10    def from_json(cls,path):
11        d=json.loads(Path(path).read_text()); req=['probe_name','n_elements','pitch_m','frequency_
12        ↳ min_hz','frequency_max_hz','center_frequency_hz']; miss=[x for x in req if x not in d]
13        if miss: raise ValueError(f'Missing transducer fields: {miss}')
14        c=cls(**{k:d[k] for k in cls.__dataclass_fields__ if k in d}); c.validate(); return c
15    def validate(self):
16        if self.n_elements!=128: raise ValueError(f'This interface requires 128 elements, got {sel
17        ↳ f.n_elements}')
18        if self.pitch_m<=0: raise ValueError('pitch_m must be positive')
19        if self.frequency_min_hz<=0 or self.frequency_max_hz<self.frequency_min_hz: raise ValueErr
20        ↳ or('Invalid frequency range')
21        if not self.frequency_min_hz<=self.center_frequency_hz<=self.frequency_max_hz: raise Value
22        ↳ Error('center_frequency_hz must be inside operating range')
```

code/src/beamformer.py

```
1 import json
2 from pathlib import Path
3 import numpy as np
4 from .transducer_interface import TransducerConfig
5
6 def create_linear_array(n_elements,pitch_m):
7     return (np.arange(n_elements,dtype=float)-(n_elements-1)/2)*pitch_m
8
9 def make_apodization(n_elements,kind='hann'):
10    k=kind.lower(); w=np.ones(n_elements) if k in ('none','uniform','rect') else np.hanning(n_elem
11    ↳ ents) if k in ('hann','hanning') else np.hamming(n_elements) if k=='hamming' else None
12    if w is None: raise ValueError(f'Unsupported apodization: {kind}')
13    return w/np.max(w) if np.max(w)>0 else w
14
15 def calculate_tx_delays(element_x_m,focus_x_m,focus_z_m,sound_speed_m_s,steering_angle_deg=0.0):
16    if sound_speed_m_s<=0 or focus_z_m<=0: raise ValueError('sound speed and focus depth must be p
17    ↳ ositive')
18    if abs(steering_angle_deg)<1e-12:
19        d=np.sqrt((focus_x_m-element_x_m)**2+focus_z_m**2); tau=(np.max(d)-d)/sound_speed_m_s
20    else:
21        theta=np.deg2rad(steering_angle_deg); tau=(element_x_m*np.sin(theta))/sound_speed_m_s; tau
22    ↳ -=tau.min()
23    return tau
24
25 def gaussian_pulse(t_s,f0_hz,sigma_cycles=1.0):
26    sigma=sigma_cycles/(2*np.pi*f0_hz); return np.exp(-0.5*(t_s/sigma)**2)*np.sin(2*np.pi*f0_hz*t
27    ↳ s)
28
29 def generate_element_pulses(delays_s,f0_hz,weights,fs_hz=None,sigma_cycles=1.0):
30    fs_hz=fs_hz or max(20*f0_hz,100e6); dur=max(6*sigma_cycles/f0_hz,2.5*2/f0_hz); t=np.arange(-du
31    ↳ r,float(np.max(delays_s))+dur,1/fs_hz); p=np.array([w*gaussian_pulse(t-d,f0_hz,sigma_cycles) for d
32    ↳ ,w in zip(delays_s,weights)]); return t,p
33
34 def build_tx_package(transducer:TransducerConfig,beam_cfg):
35    x=create_linear_array(transducer.n_elements,transducer.pitch_m); c=float(beam_cfg['sound_speed
```

```

    ↪ _m_s']]; tau=calculate_tx_delays(x,float(beam_cfg['focus_x_m']),float(beam_cfg['focus_z_m']),c,flo
    ↪ at(beam_cfg.get('steering_angle_deg',0))); w=make_apodization(transducer.n_elements,beam_cfg.get('
    ↪ apodization','hann'))
30     return {'element_x_m':x,'tx_delays_s':tau,'tx_weights':w,'tx_frequency_hz':float(transducer.ce
    ↪ nter_frequency_hz),'sound_speed_m_s':c,'focus_x_m':float(beam_cfg['focus_x_m']),'focus_z_m':float(
    ↪ beam_cfg['focus_z_m']),'steering_angle_deg':float(beam_cfg.get('steering_angle_deg',0))}
31
32 def save_package(p,out): np.savez(out,**p)
33 def save_summary(p,t,b,out):
34     s={'probe':t.probe_name,'n_elements':t.n_elements,'pitch_m':t.pitch_m,'center_frequency_hz':t.
    ↪ center_frequency_hz,'focus_x_m':b['focus_x_m'],'focus_z_m':b['focus_z_m'],'steering_angle_deg':b.g
    ↪ et('steering_angle_deg',0),'sound_speed_m_s':b['sound_speed_m_s'],'delay_min_s':float(p['tx_delays
    ↪ _s'].min()),'delay_max_s':float(p['tx_delays_s'].max()),'delay_max_us':float(p['tx_delays_s'].max(
    ↪ )*1e6)}; Path(out).write_text(json.dumps(s,indent=2))

```

code/src/acoustic_handoff.py

```

1 import numpy as np
2 REQUIRED=('element_x_m','tx_delays_s','tx_weights','tx_frequency_hz','sound_speed_m_s')
3 def load_tx_package(path):
4     with np.load(path) as d:
5         miss=[k for k in REQUIRED if k not in d]
6         if miss: raise ValueError(f'Missing TX fields: {miss}')
7         p={k:np.asarray(d[k]) for k in d.files}
8     for k in ('element_x_m','tx_delays_s','tx_weights'):
9         if p[k].shape!=(128,): raise ValueError(f'{k} must have shape (128,)'')
10    return p

```

code/run_transmit_beamforming.py

```

1 from pathlib import Path
2 import json, numpy as np
3 from src.transducer_interface import TransducerConfig
4 from src.beamformer import build_tx_package,save_package,save_summary,generate_element_pulses
5 R=Path(__file__).parent; O=R/'outputs'; O.mkdir(exist_ok=True)
6 def main():
7     t=TransducerConfig.from_json(R/'config/transducer_interface.json'); b=json.loads((R/'config/beamf
    ↪ orming.json').read_text()); p=build_tx_package(t,b); save_package(p,O/'tx_beamforming_package.npz'
    ↪ ); save_summary(p,t,b,O/'tx_beamforming_summary.json'); ts,ps=generate_element_pulses(p['tx_delays
    ↪ _s'],p['tx_frequency_hz'],p['tx_weights'],sigma_cycles=b.get('pulse_duration_sigma_cycles',1)); np
    ↪ .savez(O/'tx_element_pulses.npz',time_s=ts,pulses=ps); print('TX beamforming completed'); print('E
    ↪ lements:',t.n_elements); print('Frequency MHz:',t.center_frequency_hz/1e6); print('Delay range us:
    ↪ ',p['tx_delays_s'].min()*1e6,p['tx_delays_s'].max()*1e6); print('Handoff:',O/'tx_beamforming_packa
    ↪ ge.npz')
8 if __name__=='__main__': main()

```

code/visualize_transmit_beamforming.py

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Load transmit beamforming output
5 data = np.load("outputs/tx_beamforming_package.npz")
6
7 element_x = data["element_x_m"]
8 tx_delays = data["tx_delays_s"]
9 tx_weights = data["tx_weights"]
10
11 # Convert units for easy plotting
12 element_number = np.arange(1, len(element_x) + 1)
13 element_x_mm = element_x * 1000
14 delays_us = tx_delays * 1e6
15
16 # -----
17 # Figure 1: Transmit delay for all 128 elements
18 # -----
19
20 plt.figure(figsize=(10, 5))
21
22 plt.plot(element_number, delays_us, marker="o", markersize=3)
23
24 plt.xlabel("Transducer Element Number")
25 plt.ylabel("Transmit Delay (microseconds)")

```

```

26 plt.title("Transmit Delay for 128-Element Array")
27 plt.grid(True)
28
29 plt.tight_layout()
30 plt.show()
31
32 # -----
33 # Figure 2: Element positions and delay
34 # -----
35
36 plt.figure(figsize=(10, 5))
37
38 plt.scatter(element_x_mm, delays_us)
39
40 plt.xlabel("Element Position (mm)")
41 plt.ylabel("Transmit Delay (microseconds)")
42 plt.title("Element Position vs Transmit Delay")
43 plt.grid(True)
44
45 plt.tight_layout()
46 plt.show()
47
48 # -----
49 # Print important information
50 # -----
51
52 print("\n--- TRANSMIT BEAMFORMING SUMMARY ---")
53 print(f"Number of elements : {len(element_x)}")
54 print(f"Frequency          : {data['tx_frequency_hz'] / 1e6:.2f} MHz")
55 print(f"Focus X              : {data['focus_x_m'] * 1000:.2f} mm")
56 print(f"Focus Z              : {data['focus_z_m'] * 1000:.2f} mm")
57 print(f"Sound speed          : {data['sound_speed_m_s']:.0f} m/s")
58 print(f"Steering angle       : {data['steering_angle_deg']:.1f} degrees")
59 print(f"Minimum delay        : {delays_us.min():.4f} us")
60 print(f"Maximum delay        : {delays_us.max():.4f} us")
61
62 print("\nTransmit beamforming visualization completed.")

```

code/tests/test_beamformer.py

```

1 from pathlib import Path
2 import json, numpy as np
3 from src.transducer_interface import TransducerConfig
4 from src.beamformer import create_linear_array, calculate_tx_delays, build_tx_package
5 R=Path(__file__).parents[1]
6 def test_shapes():
7     t=TransducerConfig.from_json(R/'config/transducer_interface.json'); x=create_linear_array(128,t.p
↳ itch_m); assert x.shape==(128,); assert np.isclose(x.mean(),0); b=json.loads((R/'config/beamformin
↳ g.json').read_text()); p=build_tx_package(t,b); assert p['tx_delays_s'].shape==(128,); assert p['t
↳ x_weights'].shape==(128,)
8 def test_delay():
9     t=TransducerConfig.from_json(R/'config/transducer_interface.json'); x=create_linear_array(128,t.p
↳ itch_m); d=calculate_tx_delays(x,0,.04,1540); assert np.all(d>=0) and np.isclose(d.min(),0)

```

code/docs/RUN_ORDER.txt

1. Edit config/transducer_interface.json if the Transducer team supplies actual values.
2. Edit config/beamforming.json for focus/steering/sound speed.
3. Run: python run_transmit_beamforming.py
4. Give outputs/tx_beamforming_package.npz to Acoustic Propagation.

code/docs/ACOUSTIC_HANDOFF.md

- 1 # Acoustic hand-off
- 2 Load `outputs/tx\_beamforming\_package.npz` and use `element\_x\_m`, `tx\_delays\_s`, and `tx\_weights` d
↳ irectly. Each has shape `(128,)`. Do not create a second element geometry inside Acoustic Propagat
↳ ion. Pass `tx\_delays\_s` as the per-element transmit delay vector and apply `tx\_weights[n]` to elem
↳ ent n if apodization is supported.

code/docs/TRANSDUCER_INPUTS.md

```

1 # Transducer inputs
2 Required from Transducer team: n_elements, pitch_m, frequency_min_hz, frequency_max_hz, center_fre
  ↳ quency_hz. Optional: element width/height and material. Edit `config/transducer_interface.json`. F
  ↳ or the Philips L12-4 reference, 128 elements and 4–12 MHz are documented; pitch is not given on th
  ↳ e product page, so 0.30 mm is an editable simulation assumption.

```

The seven reference .py files in code/docs/ (Team 1 copies, not run) are not listed here to save space; they are in the folder.

run_team2.py (demo runner, written for this demo)

```

1 """
2 run_team2.py - one-command demo for Team 2: Transmit (TX) Beamforming.
3
4 What this script does (the team's own code in code/ is NOT changed):
5 1. Copies code/ into a temporary working folder and puts the two input
6   files from input/ into its config/ folder.
7 2. Runs the team's pipeline in the order they designed it:
8     python run_transmit_beamforming.py
9     python visualize_transmit_beamforming.py
10    (the second script calls plt.show(); here each figure is saved to a PNG
11     instead of opening a window).
12 3. Runs the team's tests (tests/test_beamformer.py) by importing the module
13    and calling every test_* function directly (pytest is not needed).
14 4. Prints a summary read from the team's own output files.
15 5. Makes a few extra figures using the team's own functions (file names
16    start with "extra_").
17 6. Saves everything to output/, including the console log and a
18    screenshot-style PNG of the console.
19
20 Run: python3 run_team2.py      (Windows: python run_team2.py)
21 """
22 import io
23 import json
24 import os
25 import shutil
26 import subprocess
27 import sys
28 import tempfile
29 import time
30 import traceback
31 from pathlib import Path
32
33 import numpy as np
34 import matplotlib
35
36 matplotlib.use("Agg")
37 import matplotlib.pyplot as plt # noqa: E402
38
39 HERE = Path(__file__).resolve().parent
40 CODE = HERE / "code"
41 INPUT = HERE / "input"
42 OUT = HERE / "output"
43 sys.dont_write_bytecode = True
44
45
46 # -----
47 # Console: print to screen and keep a copy for output/console_log.txt
48 # -----
49 class Tee(io.TextIOBase):
50     def __init__(self, stream):
51         self.stream, self.buf = stream, io.StringIO()
52
53     def write(self, s):
54         self.stream.write(s); self.buf.write(s); return len(s)
55
56     def flush(self):
57         self.stream.flush()
58
59
60 def banner(text):
61     print("\n" + "=" * 72 + f"\n{text}\n" + "=" * 72)
62
63
64 def run_script(cmd_args, cwd, label):
65     """Run a Python command in `cwd`, show its output, stop on failure."""

```

```

66     env = dict(os.environ, MPLBACKEND="Agg", PYTHONDONTWRITEBYTECODE="1")
67     t0 = time.time()
68     r = subprocess.run([sys.executable] + cmd_args, cwd=cwd, env=env,
69                        capture_output=True, text=True)
70     for line in r.stdout.rstrip().splitlines():
71         print(" | " + line)
72     if r.returncode != 0:
73         print(r.stderr)
74         raise SystemExit(f"ERROR: {label} failed (exit code {r.returncode}).")
75     print(f" ({label} finished in {time.time() - t0:.2f} s)")
76
77
78 # Small wrapper so the team's visualize script can run without opening windows:
79 # every plt.show() saves the current figure to a numbered PNG in output/.
80 VIS_WRAPPER = r"""
81 import sys, runpy, matplotlib
82 matplotlib.use("Agg")
83 import matplotlib.pyplot as plt
84 out_dir = sys.argv[1]
85 names = ["team_fig1_delay_vs_element.png", "team_fig2_position_vs_delay.png"]
86 count = [0]
87 def _save_instead_of_show(*a, **k):
88     i = count[0]
89     name = names[i] if i < len(names) else f"team_fig{i + 1}.png"
90     plt.gcf().savefig(out_dir + "/" + name, dpi=130)
91     plt.close("all")
92     count[0] += 1
93 plt.show = _save_instead_of_show
94 runpy.run_path("visualize_transmit_beamforming.py", run_name="__main__")
95 """
96
97 # The team's tests are plain functions; call each one and report pass/fail.
98 TEST_WRAPPER = r"""
99 import importlib, traceback
100 m = importlib.import_module("tests.test_beamformer")
101 names = sorted(n for n in dir(m) if n.startswith("test_"))
102 failed = 0
103 for n in names:
104     try:
105         getattr(m, n)(); print(f"PASS {n}")
106     except Exception as e:
107         failed += 1; print(f"FAIL {n}: {type(e).__name__}: {e}")
108 print(f"RESULT {len(names) - failed}/{len(names)} tests passed")
109 raise SystemExit(1 if failed else 0)
110 """
111
112
113 def console_to_png(text, out_png, title="Terminal", width_chars=100, max_lines=90):
114     """Draw the captured console text as a screenshot-style PNG."""
115     lines = []
116     for line in text.expandtabs(4).rstrip("\n").splitlines():
117         while len(line) > width_chars:
118             lines.append(line[:width_chars]); line = " -> " + line[width_chars:]
119         lines.append(line)
120     if len(lines) > max_lines:
121         lines = lines[:max_lines - 1] + [f"... ({len(lines) - max_lines + 1} more lines, see conso
↳ le_log.txt)"]
122     h = 0.62 + 0.165 * len(lines)
123     fig = plt.figure(figsize=(0.083 * width_chars + 0.4, h))
124     fig.patch.set_facecolor("#1e1e1e")
125     fig.text(0.012, 1 - 0.28 / h, "o o o " + title, color="#9a9a9a", fontsize=9,
126            family="DejaVu Sans Mono", va="center")
127     for i, ln in enumerate(lines):
128         fig.text(0.012, 1 - (0.55 + 0.165 * i) / h, ln, color="#e6e6e6", fontsize=8.6,
129            family="DejaVu Sans Mono", va="top")
130     fig.savefig(out_png, dpi=160, facecolor=fig.get_facecolor())
131     plt.close(fig)
132
133
134 # -----
135 # Extra figures (made by this runner with the team's own functions/outputs)
136 # -----
137 def pulse_pair_energy(dt, f0, sigma_cycles):
138     """Exact integral of p(t-a)*p(t-b) dt for the team's Gaussian pulse, dt=a-b."""
139     s = sigma_cycles / (2 * np.pi * f0); w = 2 * np.pi * f0
140     return 0.5 * s * np.sqrt(np.pi) * np.exp(-dt ** 2 / (4 * s ** 2)) * (np.cos(w * dt) - np.exp(-
↳ (w * s) ** 2))

```

```

141
142
143 def field_energy_map(x_el, delays, weights, c, f0, sigma_cycles, gx, gz):
144     """Energy of the summed pulse at every grid point (x, z).
145
146     Element n fires at delays[n]; its pulse reaches (x, z) at
147     T_n = delays[n] + r_n / c. The energy of sum_n w_n p(t - T_n) is
148     sum_n sum_m w_n w_m K(T_n - T_m) with K = pulse_pair_energy.
149     Spreading loss and element directivity are ignored (simple picture).
150     """
151     X, Z = np.meshgrid(gx, gz)
152     pts = np.column_stack([X.ravel(), Z.ravel()])
153     E = np.empty(len(pts))
154     for i in range(0, len(pts), 400):
155         p = pts[i:i + 400]
156         T = delays[None, :] + np.hypot(p[:, :1] - x_el[None, :], p[:, 1:]) / c
157         K = pulse_pair_energy(T[:, :, None] - T[:, None, :], f0, sigma_cycles)
158         E[i:i + 400] = np.einsum("n,pnm,m->p", weights, K, weights)
159     return E.reshape(X.shape)
160
161
162 def make_extra_figures(work, pkg, beam_cfg):
163     sys.path.insert(0, str(work))
164     from src.beamformer import make_apodization, gaussian_pulse, calculate_tx_delays
165
166     x = pkg["element_x_m"]; tau = pkg["tx_delays_s"]; w = pkg["tx_weights"]
167     f0 = float(pkg["tx_frequency_hz"]); c = float(pkg["sound_speed_m_s"])
168     xf = float(pkg["focus_x_m"]); zf = float(pkg["focus_z_m"])
169     sig = float(beam_cfg.get("pulse_duration_sigma_cycles", 1.0))
170     n = len(x); el = np.arange(1, n + 1)
171     made, info = [], []
172
173     # 1. Delay profile + weights actually used, plus a steering-only example
174     fig, ax = plt.subplots(1, 3, figsize=(15, 4.2))
175     ax[0].plot(el, tau * 1e6, "o-", ms=2.5, color="#1f5f99")
176     ax[0].set(title=f"Focusing delays (focus x={xf*1e3:.0f} mm, z={zf*1e3:.0f} mm)",
177              xlabel="Element number", ylabel="Delay (us)")
178     ax[0].annotate("edge elements fire first\n(delay = 0)", xy=(n, 0), xytext=(70, tau.max() * 1e6
179     * 0.08),
180                  arrowprops=dict(arrowstyle="->"), fontsize=8)
181     ax[0].annotate(f"centre fires last\n({tau.max()*1e6:.3f} us)", xy=(n / 2, tau.max() * 1e6),
182                  xytext=(n / 2 - 20, tau.max() * 1e6 * 0.35), arrowprops=dict(arrowstyle="->"),
183                  fontsize=8)
184     ax[1].plot(el, w, "o-", ms=2.5, color="#2a7a3a")
185     ax[1].set(title=f"Apodization weights used ({beam_cfg.get('apodization')})",
186              xlabel="Element number", ylabel="Weight", ylim=(-0.05, 1.1))
187     for ang in (-15, 15):
188         ax[2].plot(el, calculate_tx_delays(x, xf, zf, c, ang) * 1e6, label=f"steering {ang:+d} deg
189         ")
190     ax[2].set(title="Example only: steering delays (team function)", xlabel="Element number", ylab
191     el="Delay (us)")
192     ax[2].legend(fontsize=8)
193     for a in ax: a.grid(alpha=0.3)
194     fig.suptitle("Extra figure (made by run_team2.py from the team's output and functions)", fonts
195     ize=9, color="grey")
196     fig.tight_layout(); p = OUT / "extra_1_delays_and_weights.png"; fig.savefig(p, dpi=130); plt.c
197     lose(fig); made.append(p)
198
199     # 2. Apodization windows from make_apodization
200     fig, ax = plt.subplots(1, 3, figsize=(15, 4.2))
201     k_wave = 2 * np.pi * f0 / c
202     u_zoom = np.sin(np.radians(np.linspace(-4, 4, 1601)))
203     u_wide = np.sin(np.radians(np.linspace(-60, 60, 4801)))
204     for kind, col in (("rect", "#888888"), ("hann", "#1f5f99"), ("hamming", "#c77d0a")):
205         wk = make_apodization(n, kind)
206         ax[0].plot(el, wk, label=kind, color=col)
207         # continuous-wave far-field beam pattern (array factor) of these weights
208         af = np.abs(np.exp(1j * k_wave * np.outer(u_zoom, x)) @ wk)
209         db = 20 * np.log10(af / af.max() + 1e-12); ang = np.degrees(np.arcsin(u_zoom))
210         main = ang[db >= -6]; ic = len(db) // 2
211         j = ic
212         while j + 1 < len(db) and db[j + 1] <= db[j]:
213             j += 1
214         # walk down the main lobe to the first null
215         info.append(f"Window {kind:<8}: -6 dB main-lobe width {main.max() - main.min():.3f} deg,
216
217         f"highest side lobe {db[j:].max():.1f} dB")
218     ax[1].plot(np.degrees(np.arcsin(u_zoom)), 20 * np.log10(af / af.max() + 1e-12), label=kind

```

```

    color=col, lw=1)
211 wh = make_apodization(n, beam_cfg.get("apodization", "hann"))
212 af = np.abs(np.exp(1j * k_wave * np.outer(u_wide, x)) @ wh)
213 ax[2].plot(np.degrees(np.arcsin(u_wide)), 20 * np.log10(af / af.max() + 1e-12), color="#1f5f99")
    color=col, lw=1)
214 g_ang = np.degrees(np.arcsin(min(1.0, c / f0 / (x[1] - x[0]))))
215 for sgn in (-1, 1):
216     ax[2].annotate("grating lobe", xy=(sgn * g_ang, 0), xytext=(sgn * g_ang - 12, -25), fontsi
    ze=8,
217     arrowprops=dict(arrowstyle="→"))
218 ax[0].set(title="make_apodization(128, kind)", xlabel="Element number", ylabel="Weight")
219 ax[1].set(title=f"Beam pattern near the axis, {f0/1e6:.0f} MHz", xlabel="Angle (deg)", ylabel=
    "Level (dB)",
220     ylim=(-80, 3), xlim=(-4, 4))
221 ax[2].set(title=f"Wide view, {beam_cfg.get('apodization')} weights (pitch > wavelength/2)",
222     xlabel="Angle (deg)", ylabel="Level (dB)", ylim=(-80, 3), xlim=(-60, 60))
223 ax[0].legend(fontsize=8); ax[1].legend(fontsize=8)
224 for a in ax: a.grid(alpha=0.3)
225 fig.suptitle("Extra figure (array factor, continuous wave): rect = narrow main lobe but high s
    ide lobes; "
226     "Hann/Hamming = wider main lobe, low side lobes", fontsize=9, color="grey")
227 fig.tight_layout(); p = OUT / "extra_2_apodization_windows.png"; fig.savefig(p, dpi=130); plt.
    close(fig); made.append(p)
228
229 # 3. The pulse: one Gaussian pulse and a few element pulses from tx_element_pulses.npz
230 pul = np.load(work / "outputs" / "tx_element_pulses.npz")
231 ts, ps = pul["time_s"], pul["pulses"]
232 fig, ax = plt.subplots(1, 3, figsize=(15, 4.2))
233 t = np.linspace(-0.15e-6, 0.15e-6, 1200)
234 g = gaussian_pulse(t, f0, sig)
235 ax[0].plot(t * 1e6, g, color="#1f5f99")
236 ax[0].plot(t * 1e6, np.exp(-0.5 * (t * 2 * np.pi * f0 / sig) ** 2), "--", color="grey", label=
    "Gaussian envelope")
237 ax[0].set(title=f"gaussian_pulse(t, {f0/1e6:.0f} MHz, sigma_cycles={sig:g})", xlabel="Time (us
    ", ylabel="Amplitude")
238 ax[0].legend(fontsize=8)
239 fs = 1 / (ts[1] - ts[0])
240 spec = np.abs(np.fft.rfft(g, 65536)); fr = np.fft.rfftfreq(65536, t[1] - t[0])
241 ax[1].plot(fr / 1e6, 20 * np.log10(spec / spec.max() + 1e-12), color="#2a7a3a")
242 sdb = 20 * np.log10(spec / spec.max() + 1e-12); band = fr[sdb >= -6]
243 info.append(f" Pulse spectrum: peak at {fr[np.argmax(spec)]/1e6:.2f} MHz, "
244     f"-6 dB band {band.min()/1e6:.2f} ... {band.max()/1e6:.2f} MHz")
245 info.append(f" Grating-lobe angle for this pitch at f0: asin(wavelength/pitch) = "
246     f"{np.degrees(np.arcsin(min(1.0, c / f0 / (x[1] - x[0])))):1f} deg")
247 ax[1].axvspan(4, 12, color="orange", alpha=0.15, label="probe band 4-12 MHz")
248 ax[1].set(title="Spectrum of the pulse", xlabel="Frequency (MHz)", ylabel="dB", xlim=(0, 25),
    ylim=(-60, 3))
249 ax[1].legend(fontsize=8)
250 for k, col in zip((7, 31, 47, 63), ("b0413e", "c77d0a", "2a7a3a", "1f5f99")):
251     ax[2].plot(ts * 1e6, ps[k], color=col, lw=1, label=f"element {k+1} (delay {tau[k]*1e6:.2f}
    us)")
252 ax[2].set(title="tx_element_pulses.npz: a few element signals", xlabel="Time (us)", ylabel="We
    ighted amplitude",
253     xlim=(tau[7] * 1e6 - 0.3, tau.max() * 1e6 + 0.3))
254 ax[2].legend(fontsize=7)
255 for a in ax: a.grid(alpha=0.3)
256 fig.suptitle("Extra figure (made by run_team2.py from the team's functions and output)", font
    size=9, color="grey")
257 fig.tight_layout(); p = OUT / "extra_3_pulse.png"; fig.savefig(p, dpi=130); plt.close(fig); ma
    de.append(p)
258
259 # 4. Simple 2-D transmit field map: team delays vs. no delays
260 gx = np.arange(-10e-3, 10e-3 + 1e-9, 0.1e-3)
261 gz = np.arange(5e-3, 70e-3 + 1e-9, 0.5e-3)
262 t0 = time.time()
263 E_foc = field_energy_map(x, tau, w, c, f0, sig, gx, gz)
264 E_flat = field_energy_map(x, np.zeros_like(tau), w, c, f0, sig, gx, gz)
265 ref = max(E_foc.max(), E_flat.max())
266 fig, ax = plt.subplots(1, 3, figsize=(15, 5.2), gridspec_kw=dict(width_ratios=[1, 1, 1.1]))
267 for a, E, ttl in ((ax[0], E_foc, "With the team's TX delays"), (ax[1], E_flat, "All delays = 0
    (no focusing)")):
268     im = a.imshow(10 * np.log10(np.maximum(E, 1e-30) / ref), extent=[gx[0]*1e3, gx[-1]*1e3, gz
    [-1]*1e3, gz[0]*1e3],
269     cmap="inferno", vmin=-30, vmax=0, aspect="auto")
270 a.plot(xf * 1e3, zf * 1e3, "c+", ms=14, mew=2)
271 a.set(title=ttl, xlabel="x (mm)", ylabel="Depth z (mm)")
272 fig.colorbar(im, ax=ax[2], label="Pulse energy (dB, both maps same scale)", shrink=0.85)

```

```

273     # fine 1-D profile across the beam at the focus depth (0.005 mm steps)
274     lx = np.arange(-3e-3, 3e-3 + 1e-9, 0.005e-3)
275     L_foc = field_energy_map(x, tau, w, c, f0, sig, lx, np.array([zf]))[0]
276     L_flat = field_energy_map(x, np.zeros_like(tau), w, c, f0, sig, lx, np.array([zf]))[0]
277     lref = L_foc.max()
278     for L, lab, col in ((L_foc, "with TX delays", "#1f5f99"), (L_flat, "no delays", "#888888")):
279         ax[2].plot(lx * 1e3, 10 * np.log10(np.maximum(L, 1e-30) / lref), label=lab, color=col)
280     ax[2].axhline(-6, color="red", ls=":", lw=1, label="-6 dB")
281     ax[2].set(title=f"Across the beam at z = {zf*1e3:.1f} mm (zoom)", xlabel="x (mm)",
282             ylabel="dB (re. focus peak)", ylim=(-30, 2))
283     ax[2].grid(alpha=0.3); ax[2].legend(fontsize=8)
284     fig.suptitle("Extra figure: simple transmit-field map (sum of the delayed, weighted pulses; sp
↳ reading loss ignored)",
285             fontsize=9, color="grey")
286     p = OUT / "extra_4_tx_field_map.png"; fig.savefig(p, dpi=130); plt.close(fig); made.append(p)
287
288     # numbers from the map, for the console
289     ix0 = np.argmin(np.abs(gx - xf))
290     axis_peak_z = gz[np.argmax(E_foc[:, ix0])]
291     above = lx[L_foc / L_foc.max() >= 10 ** (-6 / 10)]
292     width6 = above.max() - above.min()
293     i0 = np.argmin(np.abs(lx - xf))
294     gain = L_foc[i0] / L_flat[i0]
295     fmap = dict(time=time.time() - t0, npts=E_foc.size * 2 + 2 * len(lx), axis_peak_z=axis_peak_z,
↳ width6=width6,
296             gain_db=10 * np.log10(gain), grid_dz=gz[1] - gz[0], grid_dx=gx[1] - gx[0])
297     return made, fmap, info
298
299
300 def main():
301     t_start = time.time()
302     if OUT.exists():
303         shutil.rmtree(OUT)
304     OUT.mkdir()
305     tee = Tee(sys.stdout); sys.stdout = tee
306     ok_tests = None
307     try:
308         banner("TEAM 2 - TRANSMIT (TX) BEAMFORMING | NITK-UsoundSim")
309         print(f"Python      : {sys.version.split()[0]} ({sys.executable})")
310         print(f"Team folder : {HERE}")
311
312         # --- Step 0: working copy -----
313         banner("STEP 0 Prepare a working copy of the team's code")
314         tmp = tempfile.TemporaryDirectory(prefix="team2_work_")
315         work = Path(tmp.name) / "Usound"
316         shutil.copytree(CODE, work, ignore=shutil.ignore_patterns("__pycache__", ".pytest_cache"))
317         shutil.rmtree(work / "outputs", ignore_errors=True) # start clean: all outputs are made
↳ fresh
318         for f in ("transducer_interface.json", "beamforming.json"):
319             shutil.copy2(INPUT / f, work / "config" / f)
320             print(f"input/{f} -> config/{f}")
321         t_cfg = json.loads((INPUT / "transducer_interface.json").read_text())
322         b_cfg = json.loads((INPUT / "beamforming.json").read_text())
323         print("Probe input      :", ", ".join(f"{k}={v}" for k, v in t_cfg.items() if k != "note
↳ s"))
324         print("Beamformer input :", ", ".join(f"{k}={v}" for k, v in b_cfg.items()))
325
326         # --- Step 1 -----
327         banner("STEP 1 Team script: python run_transmit_beamforming.py")
328         run_script(["run_transmit_beamforming.py"], work, "run_transmit_beamforming.py")
329         print(" (the 'Handoff' path above is the temporary working copy; the files are copied to
↳ output/)")
330
331         # --- Step 2 -----
332         banner("STEP 2 Team script: python visualize_transmit_beamforming.py")
333         print(" (plt.show() is replaced by 'save to PNG', so no windows open)")
334         run_script(["-c", VIS_WRAPPER, str(OUT)], work, "visualize_transmit_beamforming.py")
335
336         # --- Step 3 -----
337         banner("STEP 3 Team tests: tests/test_beamformer.py (each test_* called directly)")
338         env = dict(os.environ, PYTHONDONTWRITEBYTECODE="1")
339         r = subprocess.run([sys.executable, "-c", TEST_WRAPPER], cwd=work, env=env, capture_output
↳ =True, text=True)
340         for line in (r.stdout + r.stderr).rstrip().splitlines():
341             print(" | " + line)
342         ok_tests = r.returncode == 0
343

```

```

344 # --- Collect outputs -----
345 for f in sorted((work / "outputs").iterdir()):
346     shutil.copy2(f, OUT / f.name)
347
348 # --- Step 4: summary from the team's own files -----
349 banner("STEP 4 Summary read from the team's output files")
350 pkg = dict(np.load(OUT / "tx_beamforming_package.npz"))
351 summ = json.loads((OUT / "tx_beamforming_summary.json").read_text())
352 pul = np.load(OUT / "tx_element_pulses.npz")
353 x, tau, w = pkg["element_x_m"], pkg["tx_delays_s"], pkg["tx_weights"]
354 f0, c = float(pkg["tx_frequency_hz"]), float(pkg["sound_speed_m_s"])
355 lam = c / f0
356 first = np.flatnonzero(np.isclose(tau, tau.min())) + 1
357 last = np.flatnonzero(np.isclose(tau, tau.max())) + 1
358 rows = [
359     ("Probe", summ["probe"]),
360     ("Number of elements", f"{len(x)}"),
361     ("Pitch", f"{summ['pitch_m']*1e3:.3f} mm"),
362     ("Element x range", f"{x.min()*1e3:.2f} mm ... {x.max()*1e3:.2f} mm"),
363     ("Aperture (first to last centre)", f"{(x.max()-x.min())*1e3:.2f} mm"),
364     ("Centre frequency f0", f"{f0/1e6:.2f} MHz"),
365     ("Sound speed c", f"{c:.0f} m/s"),
366     ("Wavelength c/f0", f"{lam*1e3:.4f} mm (pitch = {summ['pitch_m']/lam:.2f} wavelengths
↳ )"),
367     ("Focus (x, z)", f"({float(pkg['focus_x_m'])*1e3:.1f} mm, {float(pkg['focus_z_m'])*1e3
↳ :.1f} mm)"),
368     ("Steering angle", f"{float(pkg['steering_angle_deg']):.1f} deg"),
369     ("F-number (focus depth / aperture)", f"{float(pkg['focus_z_m'])/(x.max()-x.min()):.2f
↳ }"),
370     ("Delay min / max", f"{tau.min()*1e6:.4f} us / {tau.max()*1e6:.4f} us"),
371     ("Elements that fire first", "", ".join(map(str, first))"),
372     ("Elements that fire last", "", ".join(map(str, last))"),
373     ("Apodization", f"{b_cfg.get('apodization')} (weights min {w.min():.4f}, max {w.max()
↳ :.4f}, "
374         f"element 1 = {w[0]:.4f}, element 64 = {w[63]:.4f})"),
375     ("Pulse file", f"pulses {pul['pulses'].shape}, time samples {pul['time_s'].shape[0]},
↳ "
376         f"fs = {1/(pul['time_s'][-1]-pul['time_s'][0])/1e6:.0f} MHz, "
377         f"t = {pul['time_s'][-1]*1e6:.3f} ... {pul['time_s'][0]*1e6:.3f} us"),
378 ]
379 for k, v in rows:
380     print(f" {k:<34}: {v}")
381 print(" Hand-off package keys (tx_beamforming_package.npz):")
382 for k, v in pkg.items():
383     print(f" {k:<20} shape {str(np.shape(v)):<8} dtype {v.dtype}")
384 sys.path.insert(0, str(work))
385 from src.acoustic_handoff import load_tx_package
386 load_tx_package(OUT / "tx_beamforming_package.npz")
387 print(" load_tx_package() check (the next team's loader): OK - all required fields, shape
↳ (128,)")
388
389 # --- Step 5: extra figures -----
390 banner("STEP 5 Extra figures (made by this runner with the team's functions)")
391 made, fmap, info = make_extra_figures(work, pkg, b_cfg)
392 print("\n".join(info))
393 for p in made:
394     print(f" saved {p.name}")
395 print(f" Field map: {fmap['npts']} grid points in {fmap['time']:.1f} s "
396       f"(grid step x {fmap['grid_dx']*1e3:.1f} mm, z {fmap['grid_dz']*1e3:.1f} mm)")
397 print(f" Field map: strongest point on the centre line at z = {fmap['axis_peak_z']*1e3:.1
↳ f} mm "
398       f"(focus set to {float(pkg['focus_z_m'])*1e3:.1f} mm)")
399 print(f" Field map: -6 dB beam width at focus depth = {fmap['width6']*1e3:.2f} mm")
400 print(f" Field map: energy at the focus is {fmap['gain_db']:.1f} dB higher than with no d
↳ elays")
401 tmp.cleanup()
402
403 # --- Done -----
404 banner("DONE")
405 print(f" Tests: {'ALL PASSED' if ok_tests else 'SOME FAILED (see STEP 3)'}")
406 print(f" Output folder: {OUT}")
407 for f in sorted(OUT.iterdir()):
408     print(f" {f.name}")
409 print(" console_log.txt")
410 print(" console_screenshot.png")
411 print(f" Total run time: {time.time() - t_start:.1f} s")
412 except SystemExit as e:

```

```
413         print(e); raise
414     except Exception:
415         traceback.print_exc(file=sys.stdout); raise
416     finally:
417         sys.stdout = tee.stream
418         text = tee.buf.getvalue()
419         (OUT / "console_log.txt").write_text(text, encoding="utf-8")
420     try:
421         console_to_png(text, OUT / "console_screenshot.png", title="python3 run_team2.py")
422     except Exception as e: # never hide the real result because of the picture
423         print("Could not draw console screenshot:", e)
424     return 0 if ok_tests else 1
425
426
427 if __name__ == "__main__":
428     sys.exit(main())
```

Line numbers in these listings are the real line numbers of each file (a long line that continues on the next printed line is marked with an arrow and has no number of its own).

4 Input

The sample input used by the run

The run uses two small JSON files in **input/**. They are exact copies of the team's code/config/ files. run_team2.py copies them into the working copy's config/ before running, so you can change the input without touching the team's code.

input/transducer_interface.json (probe)

Field	Value	Meaning
probe_name	Philips L12-4 reference model	reference probe
n_elements	128	must be 128
pitch_m	0.0003 (0.30 mm)	element spacing (editable assumption)
frequency_min_hz / max_hz	4000000 / 12000000	operating band 4 to 12 MHz
center_frequency_hz	8000000	8 MHz centre
element_width_m / height_m	null / null	unknown, not used
material	PZT-5H	piezo material (not used in maths)

input/beamforming.json (beam settings)

Field	Value	Meaning
focus_x_m	0.0	focus straight in front of the centre
focus_z_m	0.04	focus depth 40 mm
steering_angle_deg	0.0	no steering
sound_speed_m_s	1540.0	soft-tissue sound speed
apodization	hann	Hann weights
pulse_cycles	2.0	not used by the code
pulse_duration_sigma_cycles	1.0	sets the Gaussian width σ

Try it

Change focus_z_m to 0.02 (20 mm) and run again: the delay curve becomes deeper (bigger delays) and the bright spot in extra_4 moves up.

Change apodization to "rect": all weights become 1.

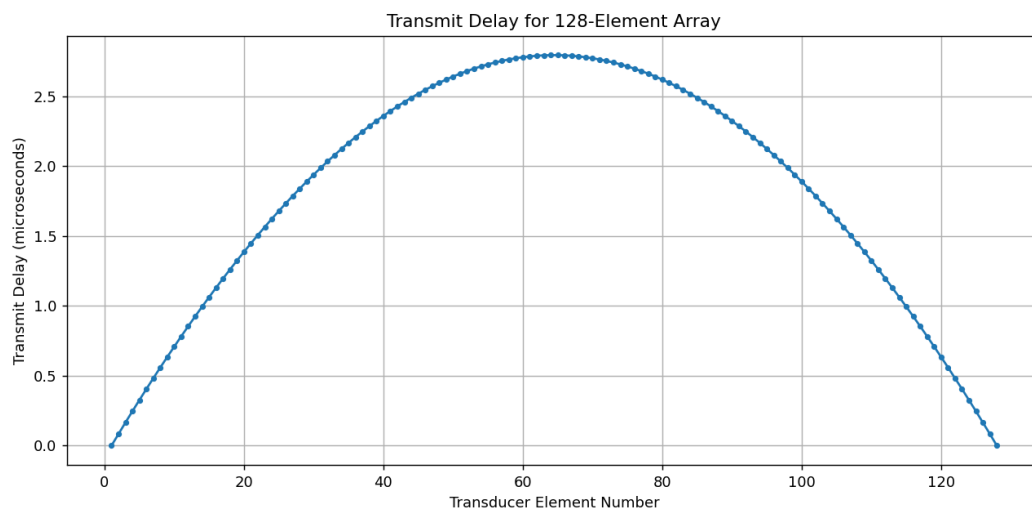
5 Output

Every file produced by run_team2.py, with the real console output

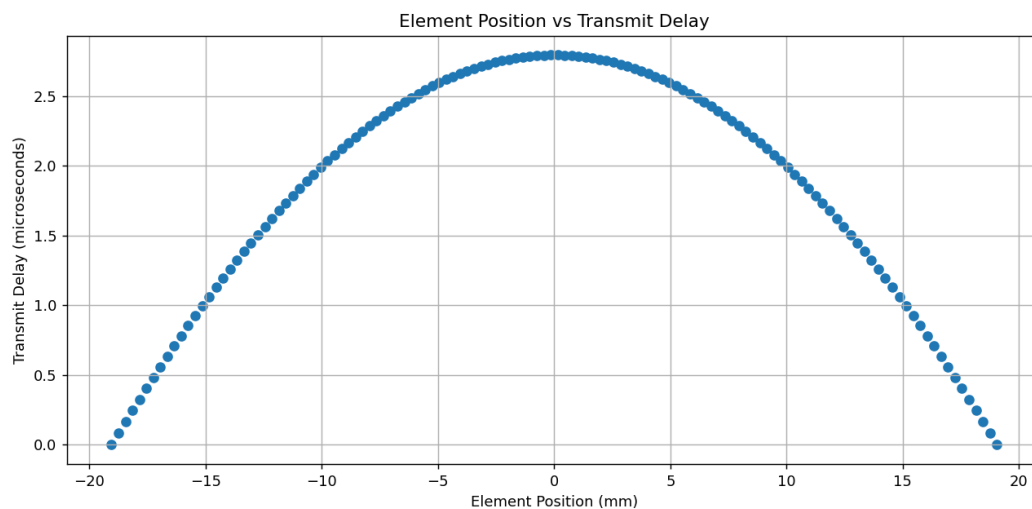
5.1 Data files (made by the team's code)

File	Contents (from this run)
tx_beamforming_package.npz	The hand-off. element_x_m, tx_delays_s, tx_weights (128 each), tx_frequency_hz = 8e6, sound_speed_m_s = 1540, focus_x_m = 0, focus_z_m = 0.04, steering_angle_deg = 0.
tx_beamforming_summary.json	delay_min_s = 0.0, delay_max_us = 2.795048, plus probe and focus.
tx_element_pulses.npz	time_s and pulses: pulses (128, 688), time samples 688, fs = 160 MHz, t = -0.750 ... 3.544 us

5.2 Figures made by the team's script

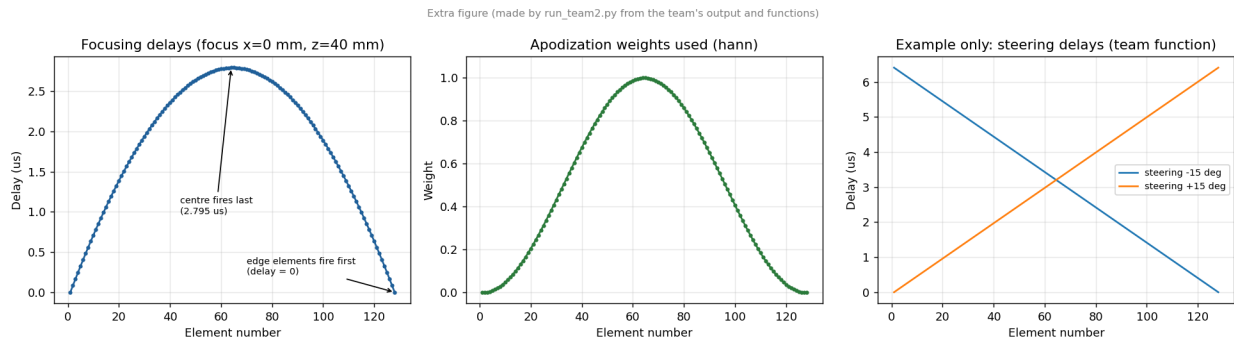


team_fig1: delay of every element. It is an upside-down bowl: elements 1 and 128 have 0 us (fire first), the centre elements have 2.7950 us (fire last). It is symmetric because the focus is at $x = 0$.

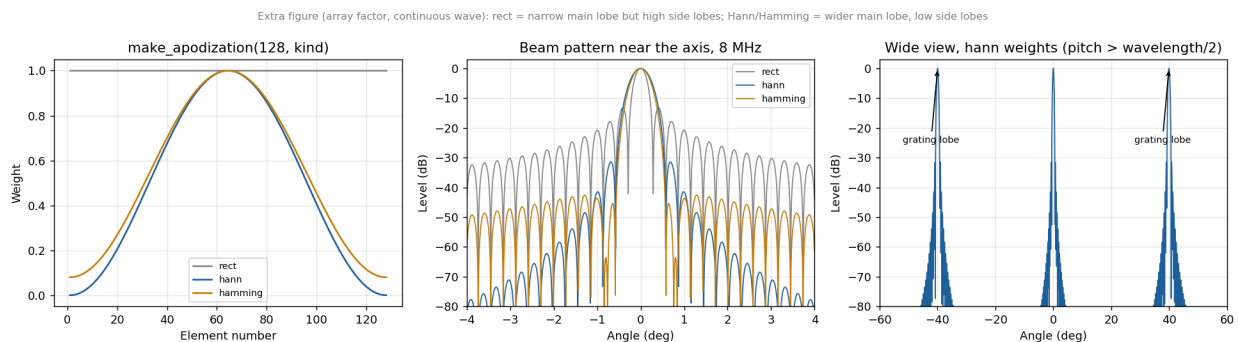


team_fig2: the same delays against element position in mm (from -19.05 mm to +19.05 mm). Its shape follows the hyperbola-like path difference d_n .

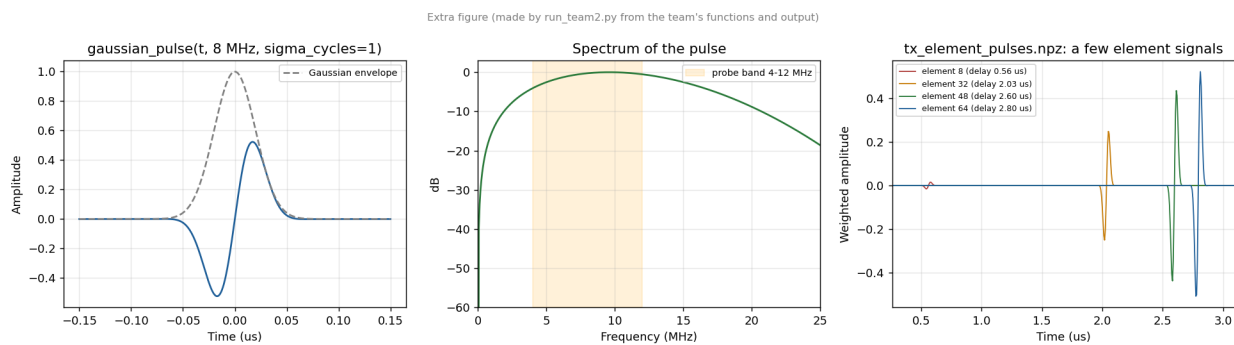
5.3 Extra figures made by run_team2.py (using the team's functions and outputs)



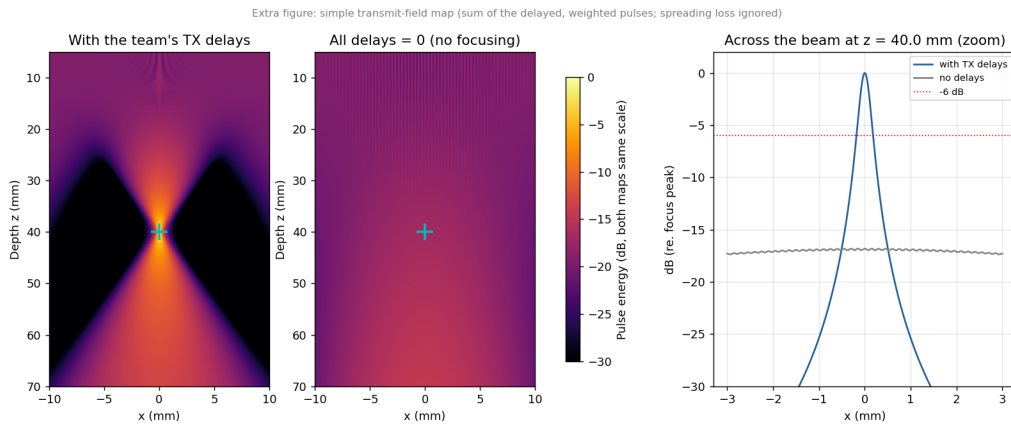
extra_1: left, the focusing delays (edge first, centre last). Middle, the Hann weights actually used (0 at both ends, 1 in the middle). Right, only as an example, calculate_tx_delays with steering -15 and +15 degrees: a straight ramp (focus not used when steering).



extra_2: left, the three windows from make_apodization. Middle, their beam patterns near the axis: rect has the narrowest main lobe but side lobes at -13.3 dB; Hann -31.5 dB; Hamming -42.6 dB. Right, wide view with Hann weights: grating lobes near ± 39.9 degrees because the pitch is 1.56 wavelengths.



extra_3: left, gaussian_pulse at 8 MHz with sigma_cycles = 1 (about one oscillation long). Middle, its spectrum (peak 9.57 MHz, wide band). Right, four rows from tx_element_pulses.npz: each element's pulse starts at its own delay, and elements near the edge are weaker because of the Hann weight.



extra_4: sum of the delayed, weighted pulses at each point (energy in dB, same scale in both maps). With the team's delays the energy is highest at the focus (strongest point on the centre line at $z = 40.0$ mm); with zero delays there is no focus. Right: across the beam at 40 mm the -6 dB width is 0.36 mm, and the focus is 16.8 dB stronger than without delays.

5.4 Console output (real, from output/console_log.txt)

python3 run_team2.py

```
=====
TEAM 2 - TRANSMIT (TX) BEAMFORMING | NITK-UsoundSim
=====
Python      : 3.12.4 (/usr/local/bin/python3)
Team folder : /Users/darshil/Downloads/NITK-UsoundSim_Team_Projects/Team_2_TX_Beamforming

=====
STEP 0 Prepare a working copy of the team's code
=====
input/transducer_interface.json -> config/transducer_interface.json
input/beamforming.json -> config/beamforming.json
Probe input      : probe_name=Philips L12-4 reference model, n_elements=128, pitch_m=0.0003, frequency
↳ _min_hz=4000000.0, frequency_max_hz=12000000.0, center_frequency_hz=8000000.0, element_width_m=None,
↳ element_height_m=None, material=PZT-5H
Beamformer input : focus_x_m=0.0, focus_z_m=0.04, steering_angle_deg=0.0, sound_speed_m_s=1540.0, apod
↳ ization=hann, pulse_cycles=2.0, pulse_duration_sigma_cycles=1.0

=====
STEP 1 Team script: python run_transmit_beamforming.py
=====
| TX beamforming completed
| Elements: 128
| Frequency MHz: 8.0
| Delay range us: 0.0 2.7950482224488855
| Handoff: /private/var/folders/50/33vqm1x5yd4znrpykvksbpc0000gn/T/team2_work_bk28vq70/Usound/outputs
↳ /tx_beamforming_package.npz
(run_transmit_beamforming.py finished in 0.27 s)
(the 'Handoff' path above is the temporary working copy; the files are copied to output/)

=====
STEP 2 Team script: python visualize_transmit_beamforming.py
=====
(plt.show() is replaced by 'save to PNG', so no windows open)
|
| --- TRANSMIT BEAMFORMING SUMMARY ---
| Number of elements : 128
| Frequency          : 8.00 MHz
| Focus X            : 0.00 mm
| Focus Z            : 40.00 mm
| Sound speed        : 1540 m/s
| Steering angle     : 0.0 degrees
| Minimum delay      : 0.0000 us
| Maximum delay      : 2.7950 us
|
| Transmit beamforming visualization completed.
(visualize_transmit_beamforming.py finished in 0.63 s)
```

```
=====
STEP 3 Team tests: tests/test_beamformer.py (each test_* called directly)
=====
```

```
| PASS test_delay
| PASS test_shapes
| RESULT 2/2 tests passed
```

```
=====
STEP 4 Summary read from the team's output files
=====
```

```
Probe : Philips L12-4 reference model
Number of elements : 128
Pitch : 0.300 mm
Element x range : -19.05 mm ... 19.05 mm
Aperture (first to last centre) : 38.10 mm
Centre frequency f0 : 8.00 MHz
Sound speed c : 1540 m/s
Wavelength c/f0 : 0.1925 mm (pitch = 1.56 wavelengths)
Focus (x, z) : (0.0 mm, 40.0 mm)
Steering angle : 0.0 deg
F-number (focus depth / aperture) : 1.05
Delay min / max : 0.0000 us / 2.7950 us
Elements that fire first : 1, 128
Elements that fire last : 61, 62, 63, 64, 65, 66, 67, 68
Apodization : hann (weights min 0.0000, max 1.0000, element 1 = 0.0000, element
↳ 64 = 1.0000)
Pulse file : pulses (128, 688), time samples 688, fs = 160 MHz, t = -0.750 ...
↳ 3.544 us
Hand-off package keys (tx_beamforming_package.npz):
  element_x_m shape (128,) dtype float64
  tx_delays_s shape (128,) dtype float64
  tx_weights shape (128,) dtype float64
  tx_frequency_hz shape () dtype float64
  sound_speed_m_s shape () dtype float64
  focus_x_m shape () dtype float64
  focus_z_m shape () dtype float64
  steering_angle_deg shape () dtype float64
load_tx_package() check (the next team's loader): OK - all required fields, shape (128,)
```

```
=====
STEP 5 Extra figures (made by this runner with the team's functions)
=====
```

```
Window rect : -6 dB main-lobe width 0.340 deg, highest side lobe -13.3 dB
Window hann : -6 dB main-lobe width 0.570 deg, highest side lobe -31.5 dB
Window hamming : -6 dB main-lobe width 0.520 deg, highest side lobe -42.6 dB
Pulse spectrum: peak at 9.57 MHz, -6 dB band 3.11 ... 18.05 MHz
Grating-lobe angle for this pitch at f0: asin(wavelength/pitch) = 39.9 deg
saved extra_1_delays_and_weights.png
saved extra_2_apodization_windows.png
saved extra_3_pulse.png
saved extra_4_tx_field_map.png
Field map: 55064 grid points in 13.8 s (grid step x 0.1 mm, z 0.5 mm)
Field map: strongest point on the centre line at z = 40.0 mm (focus set to 40.0 mm)
Field map: -6 dB beam width at focus depth = 0.36 mm
Field map: energy at the focus is 16.8 dB higher than with no delays
```

```
=====
DONE
=====
```

```
Tests: ALL PASSED
Output folder: /Users/darshil/Downloads/NITK-UsoundSim_Team_Projects/Team_2_TX_Beamforming/output
  extra_1_delays_and_weights.png
  extra_2_apodization_windows.png
  extra_3_pulse.png
  extra_4_tx_field_map.png
  team_fig1_delay_vs_element.png
  team_fig2_position_vs_delay.png
  tx_beamforming_package.npz
  tx_beamforming_summary.json
  tx_element_pulses.npz
  console_log.txt
  console_screenshot.png
Total run time: 15.8 s
```

The same text is saved as a picture in output/console_screenshot.png:

```
o o o  python3 run_team2.py

=====
TEAM 2 - TRANSMIT (TX) BEAMFORMING | NITK-UsoundSim
=====
Python      : 3.12.4 (/usr/local/bin/python3)
Team folder : /Users/darshil/Downloads/NITK-UsoundSim_Team_Projects/Team_2_TX_Beamforming
=====

STEP 0 Prepare a working copy of the team's code
=====
input/transducer_interface.json -> config/transducer_interface.json
input/beamforming.json -> config/beamforming.json
Probe input      : probe_name=Philips L12-4 reference model, n_elements=128, pitch_m=0.0003, frequ
-> ency_min_hz=4000000.0, frequency_max_hz=12000000.0, center_frequency_hz=8000000.0, element_wid
-> h_m=None, element_height_m=None, material=PZT-5H
Beamformer input : focus_x_m=0.0, focus_z_m=0.04, steering_angle_deg=0.0, sound_speed_m_s=1540.0,
-> apodization=hann, pulse_cycles=2.0, pulse_duration_sigma_cycles=1.0

=====

STEP 1 Team script: python run_transmit_beamforming.py
=====
| TX beamforming completed
| Elements: 128
| Frequency MHz: 8.0
| Delay range us: 0.0 2.7950482224488855
| Handoff: /private/var/folders/50/33vqmx1x5yd4znrpykvksbpc0000gn/T/team2_work_bk28vq70/Usound/out
-> puts/tx_beamforming_package.npz
(run transmit beamforming.py finished in 0.27 s)
(the 'Handoff' path above is the temporary working copy; the files are copied to output/)

=====

STEP 2 Team script: python visualize_transmit_beamforming.py
=====
(plt.show() is replaced by 'save to PNG', so no windows open)
|
| --- TRANSMIT BEAMFORMING SUMMARY ---
| Number of elements : 128
| Frequency           : 8.00 MHz
| Focus X             : 0.00 mm
| Focus Z             : 40.00 mm
| Sound speed         : 1540 m/s
| Steering angle      : 0.0 degrees
| Minimum delay       : 0.0000 us
| Maximum delay       : 2.7950 us
|
| Transmit beamforming visualization completed.
(visualize_transmit_beamforming.py finished in 0.63 s)

=====

STEP 3 Team tests: tests/test_beamformer.py (each test_* called directly)
=====
| PASS test_delay
| PASS test_shapes
| RESULT 2/2 tests passed

=====

STEP 4 Summary read from the team's output files
=====
Probe                : Philips L12-4 reference model
Number of elements   : 128
Pitch                : 0.300 mm
Element x range       : -19.05 mm ... 19.05 mm
Aperture (first to last centre) : 38.10 mm
Centre frequency f0   : 8.00 MHz
Sound speed c         : 1540 m/s
Wavelength c/f0      : 0.1925 mm (pitch = 1.56 wavelengths)
Focus (x, z)          : (0.0 mm, 40.0 mm)
Steering angle        : 0.0 deg
F-number (focus depth / aperture) : 1.05
Delay min / max       : 0.0000 us / 2.7950 us
Elements that fire first : 1, 128
Elements that fire last  : 61, 62, 63, 64, 65, 66, 67, 68
Apodization           : hann (weights min 0.0000, max 1.0000, element 1 = 0.0000, ele
-> ment 64 = 1.0000)
Pulse file            : pulses (128, 688), time samples 688, fs = 160 MHz, t = -0.750
-> ... 3.544 us
Hand-off package keys (tx_beamforming_package.npz):
  element_x_m         shape (128,) dtype float64
  tx_delays_s          shape (128,) dtype float64
  tx_weights           shape (128,) dtype float64
  tx_frequency_hz      shape ()    dtype float64
  sound_speed_m_s      shape ()    dtype float64
  focus_x_m            shape ()    dtype float64
  focus_z_m            shape ()    dtype float64
  steering_angle_deg   shape ()    dtype float64
load_tx_package() check (the next team's loader): OK - all required fields, shape (128,)

=====

STEP 5 Extra figures (made by this runner with the team's functions)
... (32 more lines, see console_log.txt)
```

output/console_screenshot.png

5.5 What the results tell us

- The delays range from 0 to 2.7950 us, matching the hand calculation in 2.2.
- Firing order: elements 1 and 128 first, elements 61 to 68 last (their delays are equal to 4 decimals of a microsecond).
- The team's own loader `load_tx_package()` accepts the package, so the hand-off format is correct.
- The field map confirms the design: the energy peak sits at the chosen 40 mm depth.

Honest observations

`pulse_cycles` in `beamforming.json` is not used by any team code.

When `steering_angle_deg` is not 0, the code gives steering-only delays and ignores the focus (no combined steer + focus).

With Hann weights, the two edge elements get weight exactly 0.

The 0.30 mm pitch (1.56 wavelengths at 8 MHz) allows grating lobes near 40 degrees; the team marks it as an editable assumption.

`sigma_cycles = 1` gives a very short pulse whose -6 dB band (about 3 to 18 MHz) is wider than the 4 to 12 MHz probe band.

`visualize_transmit_beamforming.py` uses a relative path and `plt.show()`, so it must be run from the team folder (the runner handles this).

6 How to run

Step-by-step for macOS, Linux and Windows

1. Install **Python 3.10 or newer** from python.org. On Windows tick 'Add Python to PATH'.
2. Open a terminal: macOS Terminal, Linux terminal, or Windows Command Prompt / PowerShell.
3. Go into the folder: `cd path/to/Team_2_TX_Beamforming`
4. Install packages once: macOS/Linux `python3 -m pip install -r requirements.txt`;
Windows `python -m pip install -r requirements.txt`
5. Run: macOS/Linux `python3 run_team2.py`; Windows `python run_team2.py`
6. Look in **output/** for the PNG figures, the .npz/.json files, console_log.txt and console_screenshot.png.
7. Optional, the team's tests with pytest: `cd code && python3 -m pytest tests` (runs the team's 2 tests; needs pytest from requirements.txt).

Expected runtime: about 20 seconds (measured 18.8 s to 21.8 s on a shared 8-core Mac). The team's two scripts take under 1 second; most time is the extra field map. No windows open; all figures are saved.

To run the team's scripts by hand (their original way), open a terminal in **code/** and type `python3 run_transmit_beamforming.py` then `python3 visualize_transmit_beamforming.py` (this opens two plot windows and writes to code/outputs/).

Troubleshooting

Problem	Fix
python3: command not found / python is not recognized	Install Python 3.10+; on Windows use python or py.
No module named 'numpy' or 'matplotlib'	Run the pip command (step 4) with the same Python.
TypeError: unsupported operand type(s) for	Python is older than 3.10; install a newer one.
ValueError: This interface requires 128 elements	Keep <code>n_elements = 128</code> in <code>input/transducer_interface.json</code> .
ValueError: center_frequency_hz must be inside operating range	Keep the centre frequency between min and max.
ValueError: sound speed and focus depth must be positive	Use <code>focus_z_m > 0</code> and <code>sound_speed_m_s > 0</code> .
PermissionError writing output/	Copy the folder somewhere you can write (e.g. Desktop).

7 Check yourself

Viva-style questions with short answers

Why do the edge elements fire first when focusing?

They are farther from the focus, so their sound needs more time. Firing them first makes all waves arrive at the focus together.

Write the focusing delay formula and explain each term.

$\tau_n = (\max d - d_n)/c$ with $d_n = \sqrt{(x_n - x_f)^2 + z_f^2}$: d_n is the path from element n to the focus, c the sound speed; the farthest element gets delay 0.

Why is the maximum delay 2.795 us here?

Edge path 44.305 mm minus centre path 40 mm = 4.305 mm; $4.305 \text{ mm} / 1540 \text{ m/s} = 2.795 \text{ us}$.

What does apodization trade off?

Main-lobe width against side-lobe level. Rect: narrowest main lobe, side lobes -13.3 dB. Hann: wider main lobe, side lobes -31.5 dB.

What is inside the TX package and who uses it?

`element_x_m`, `tx_delays_s`, `tx_weights` (128 each), `tx_frequency_hz`, `sound_speed_m_s`, `focus` and `steering`. The Acoustic Propagation team loads it with `load_tx_package()`.

What happens to the delays if the steering angle is 15 degrees in this code?

They become a straight ramp $x_n \sin(\theta)/c$ shifted to start at 0; the focus is not used in that case.